

Automatic DOCX Report Download Guide

Overview

The Flask backend now supports automatic `.docx` report download immediately after an audit completes. This eliminates the need for a separate API call to generate the report.

Backend Changes

Enhanced `/api/audit` Endpoint

The `/api/audit` endpoint now accepts a `format` query parameter:

Query Parameters:

- `format` : `"json"` (default) or `"docx"`

Usage Examples

1. Default JSON Response (Backward Compatible)

```
// POST /api/audit
fetch('http://localhost:5000/api/audit', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    url: 'https://example.com',
    packs: ['base'],
    plan: 'quickscan'
  })
})
.then(response => response.json())
.then(data => {
  console.log('Audit results:', data);
  // Handle JSON response
});
```

2. Automatic DOCX Download

```
// POST /api/audit?format=docx
fetch('http://localhost:5000/api/audit?format=docx', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    url: 'https://example.com',
    packs: ['base'],
    plan: 'quicksan'
  })
})
.then(response => response.blob())
.then(blob => {
  // Create download link
  const url = window.URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = 'audit_report.docx';
  document.body.appendChild(a);
  a.click();
  window.URL.revokeObjectURL(url);
  document.body.removeChild(a);
});
```

3. React/Next.js Example with Auto-Download

```

// components/AuditRunner.tsx
'use client'

import { useState } from 'react'

export default function AuditRunner() {
  const [isRunning, setIsRunning] = useState(false)
  const [url, setUrl] = useState('')

  const runAuditAndDownload = async () => {
    setIsRunning(true)

    try {
      // Run audit with DOCX format
      const response = await fetch('http://localhost:5000/api/audit?format=docx', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json'
        },
        body: JSON.stringify({
          url: url,
          packs: ['base'],
          plan: 'quickscan'
        })
      })

      if (!response.ok) {
        throw new Error('Audit failed')
      }

      // Get the blob
      const blob = await response.blob()

      // Trigger download
      const downloadUrl = window.URL.createObjectURL(blob)
      const a = document.createElement('a')
      a.href = downloadUrl

      // Extract filename from Content-Disposition header if available
      const contentDisposition = response.headers.get('Content-Disposition')
      const filename = contentDisposition
        ? contentDisposition.split('filename=')[1]?.replace(/"/g, '')
        : 'audit_report.docx'

      a.download = filename
      document.body.appendChild(a)
      a.click()
      window.URL.revokeObjectURL(downloadUrl)
      document.body.removeChild(a)

      alert('Report downloaded successfully!')
    } catch (error) {
      console.error('Audit error:', error)
      alert('Failed to run audit')
    } finally {
      setIsRunning(false)
    }
  }

  return (
    <div>
      <input

```

```

    type="url"
    value={url}
    onChange={(e) => setUrl(e.target.value)}
    placeholder="https://example.com"
    className="border p-2 rounded mr-2"
  />
  <button
    onClick={runAuditAndDownload}
    disabled={isRunning || !url}
    className="bg-blue-600 text-white px-4 py-2 rounded disabled:opacity-50"
  >
    {isRunning ? 'Running Audit...' : 'Run Audit & Download Report'}
  </button>
</div>
)
}

```

Response Details

JSON Response (format=json or default)

```

{
  "success": true,
  "data": {
    "audit_id": 123,
    "url": "https://example.com",
    "domain": "example.com",
    "page_count": 5,
    "packs": ["base"],
    "plan": "quickscan",
    "scores": {
      "overall": 85.5,
      "by_category": {
        "Technical": 90.0,
        "Content": 80.0
      }
    },
    "findings": [...]
  }
}

```

DOCX Response (format=docx)

Headers:

```

Content-Type: application/vnd.openxmlformats-officedocument.wordprocessingml.document
Content-Disposition: attachment; filename="example.com_audit_20251113.docx"

```

Body: Binary .docx file stream

Testing

A test script is available at `/home/ubuntu/github_repos/vysalytica/test_docx_download.py` :

```
# Start Flask server (in one terminal)
cd /home/ubuntu/github_repos/vysalytica/api
python api.py

# Run tests (in another terminal)
cd /home/ubuntu/github_repos/vysalytica
python test_docx_download.py
```

Benefits

1. **Single API Call:** No need to call `/api/report/docx` separately
2. **Automatic Download:** Browser automatically prompts for download
3. **Backward Compatible:** Existing JSON-based implementations continue to work
4. **Proper Headers:** `Content-Disposition: attachment` ensures browser downloads the file

Migration Path

Existing code using the separate `/api/report/docx` endpoint can continue to work. New implementations can use the simplified `?format=docx` approach for a streamlined experience.

Frontend Integration Checklist

- ☐ Update audit button to call `/api/audit?format=docx`
- ☐ Handle blob response instead of JSON
- ☐ Implement automatic download trigger
- ☐ Add loading state during audit
- ☐ Handle errors appropriately
- ☐ Test with different browsers
- ☐ Update user feedback messages

Browser Compatibility

The download functionality works in all modern browsers:

- ☒ Chrome/Edge (Chromium)
- ☒ Firefox
- ☒ Safari
- ☒ Mobile browsers

Security Considerations

- Rate limiting applies to both JSON and DOCX responses
- API key requirements remain unchanged for non-quickscan plans
- CORS settings apply to both response formats